

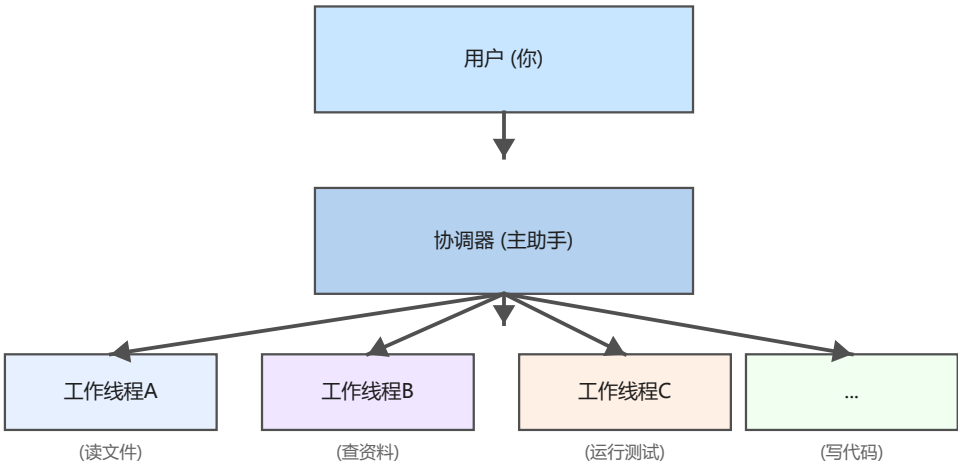
大模型异常降级熔断策略深度分析

分析日期：2026-07-09 项目：Claude Code（源码快照）

目录

- 1. 整体架构概览
- 2. API 通信可靠性
- 3. 上下文窗口管理
- 4. Agent 编排与隔离
- 5. 错误处理与恢复
- 6. 进程级保障
- 7. 熔断与降级模式全景

1. 整体架构概览



1.1 为什么需要降级熔断策略？

大模型 API 是一个不可靠的外部依赖。在实际使用中，可能遇到以下异常：

- 服务器过载（529）：API 服务器容量不足，请求被拒绝
- 速率限制（429）：账户配额或速率超限
- 上下文超限（400）：对话太长，超过模型上下文窗口
- 连接超时/断开：网络不稳定或代理问题
- 认证错误（401/403）：令牌过期或权限变更
- 模型容量不足：Opus 等高容量模型因负载被拒绝
- 未知错误（5xx）：服务器内部错误

如果不对这些异常做处理，一次 API 失败就会导致整个会话中断，用户体验极差。Claude Code 设计了多层防御体系来应对这些异常：



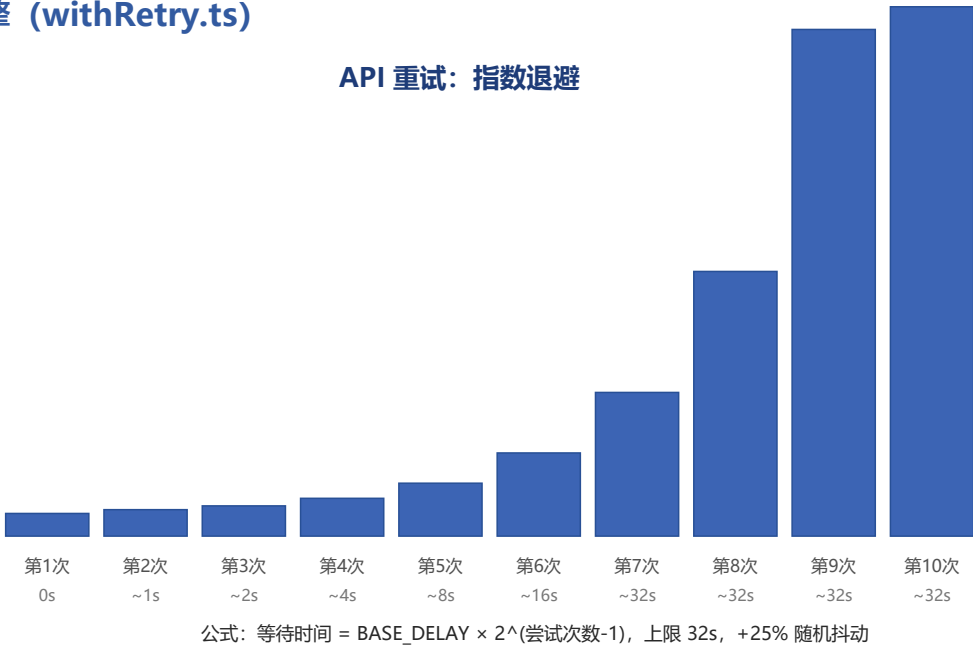
1.2 核心设计哲学

Claude Code 的降级熔断策略遵循以下原则：

原则	说明	体现
自动恢复优先	尽可能自动重试和恢复，减少用户介入	10 次重试、指数退避、持久重试模式
渐进式降级	降级不是全有全无，是逐步降低服...	快速模式 → 标准模式 → 备选模型 ...
熔断保护	连续失败时停止重试，避免加剧问题	3 次 529 触发模型回退、3 次压缩...
优雅降级	降级时提供清晰的用户反馈	每条错误消息都包含可操作的建议
隔离故障	一个子任务的失败不影响主任务	子 AbortController、Agent 隔离

2. API 通信可靠性

2.1 重试引擎 (withRetry.ts)



为什么需要重试引擎？

大模型

API

是典型的"尽力而为"服务，一次请求失败不一定代表永远失败——可能是服务器正在重启、负载暂时波动或网络抖动。盲目重试可能让过载更严重，不重试则让可用性降低。Claude Code 的重试引擎在两者之间找到平衡。

核心参数

参数	默认值	说明
`DEFAULT_MAX_RETRIES`	10	最大重试次数（可被 `CLAUDE_CO...
`BASE_DELAY_MS`	500ms	指数退避的基准延迟
`MAX_529_RETRIES`	3	连续 529 错误的阈值，超过触发模...
`PERSISTENT_MAX_BACKOFF_MS`	5 分钟	持久重试模式下的最大退避间隔
`PERSISTENT_RESET_CAP_MS`	6 小时	持久重试模式的绝对上限
`HEARTBEAT_INTERVAL_MS`	30 秒	持久重试模式的心跳间隔
`SHORT_RETRY_THRESHOLD_MS`	20 秒	区分"短等待"和"快速模式冷却"的阈值
`MIN_COOLDOWN_MS`	10 分钟	快速模式冷却的最小持续时间

重试流程

```
withRetry() 循环（最多 maxRetries + 1 次尝试）
|
├─ 1. 检查 AbortSignal → 已中止则抛出 APIUserAbortError
|
├─ 2. 检查模拟速率限制 → Ant 员工测试专用
|
├─ 3. 需要重新认证 → 401/403/ECONNRESET/EPIPE 时清除缓存
|
├─ 4. 执行操作 → 成功则返回值
|
├─ 5. 捕获错误 → 按优先级处理的降级策略：
|   |
|   └─ A. 快速模式 429/529（非持久模式）
|       └─ 额度超限 → 永久禁用快速模式，继续重试
|       └─ 短等待（<20s）→ 休眠后保留快速模式重试
|       └─ 长等待/未知 → 触发快速模式冷却（至少 10min）
|   |
|   └─ B. 快速模式被拒绝（400 "Fast mode is not enabled"）
|       → 永久禁用快速模式
|   |
|   └─ C. 非前台 529 → 立即抛出 CannotRetryError
|       （防止后台任务在容量级联时放大流量）
|   |
|   └─ D. 连续 529 追踪
|       └─ < 3 次 → 继续重试
|       └─ >= 3 次 + 有备选模型 → 抛出 FallbackTriggeredError
|       └─ >= 3 次 + 无备选 + 非持久 → 抛出 CannotRetryError
|   |
|   └─ E. 重试耗尽（attempt > maxRetries）→ 抛出 CannotRetryError
|   |
|   └─ F. 云认证错误（AWS/GCP）→ 清除凭据缓存后重试
|   |
|   └─ G. 不可重试错误 → 抛出 CannotRetryError
|   |
|   └─ H. 上下文超限（400）→ 调整 max_tokens 后重试
|   |
|   └─ I. 正常重试
|       └─ 持久模式：使用 persistentAttempt 计数，最长 6 小时
|       └─ 普通模式：指数退避 + 抖动
```

指数退避与抖动

```
// 退避计算 (withRetry.ts: getRetryDelay)
delay = min(BASE_DELAY_MS * 2^(attempt-1), maxDelayMs) + random(0, 0.25 * BASE_DELAY_MS)
```

为什么需要指数退避？

如果每次重试等同样长的时间，服务器可能还在过载，重试也是白费。指数增长让重试间隔迅速扩大，给服务器恢复的时间。

为什么需要抖动？

如果所有客户端用同样的退避策略，它们会在同一时刻重试，这就是"惊群效应"。随机抖动分摊了重试压力。

529 重试的"前台/后台"区分

为什么需要区分？

想象一个场景：API

正在经历容量危机。如果所有查询都疯狂重试，流量会放大到正常值的数十倍，让问题更严重。

```
const FOREGROUND_529_RETRY_SOURCES = new Set([
  'repl_main_thread', 'sdk', 'agent:*', 'compact',
  'hook_agent', 'hook_prompt', 'verification_agent',
  'side_question', 'auto_mode', 'bash_classifier'
])
```

只有前台查询（用户直接发起的对话、Agent

任务、压缩等）会重试

529。后台任务（摘要生成、标题建议、分类器）遇到 529 直接放弃，防止容量级联故障。

2.2 模型回退策略

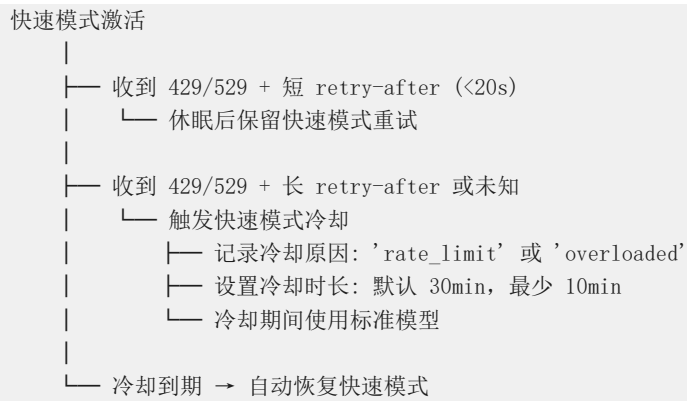
为什么需要模型回退？当 Opus 等高容量模型持续过载时，继续等待只会浪费用户的耐心。切换到 Sonnet 等更轻量的模型可以快速恢复服务。

连续 529 触发模型回退

```
连续 529 计数
|
|— 第 1 次 529 → 记录计数 = 1，继续重试
|— 第 2 次 529 → 计数 = 2，继续重试
|— 第 3 次 529 → 计数 = 3
|   |— 有 fallbackModel → 抛出 FallbackTriggeredError
|   |   |— 记录 originalModel 和 fallbackModel
|   |   |— 上层切换模型后重试
|   |— 无 fallbackModel → 抛出 CannotRetryError
|   |   |— 显示 "Repeated 529 Overloaded errors"
|
|— 任意成功 → 重置计数 = 0
```

FallbackTriggeredError 携带 originalModel 和 fallbackModel 信息，上层可以据此切换到备选模型后重试。这个机制在 Opus → Sonnet 的降级场景中特别有用。

快速模式冷却 (Circuit Breaker)



这就像家里的电闸：电流过大时跳闸保护电路，冷却一段时间后可以重新合闸。不同的是，Claude Code 的"电闸"会自动合闸——冷却到期后快速模式自动恢复。

2.3 持久重试模式

为什么需要持久重试？

在无人值守模式（CI/CD、批量处理）下，手动重试不可能。系统需要自动持续重试，直到成功或绝对超时。

由 `CLAUDE_CODE_UNATTENDED_RETRY` 环境变量启用：

- 429/529 错误无限重试——attempt 计数器卡在 maxRetries 不再增长
- 使用独立的 persistentAttempt 计数器计算退避（最长 5 分钟）
- 绝对上限 6 小时：PERSISTENT_RESET_CAP_MS
- 长等待被切分为 30 秒心跳段，定期输出 SystemAPIErrorMessage 以保持连接活跃
- 窗口型速率限制：读取 anthropic-ratelimit-unified-reset 头部，精确等待到重置时间

2.4 错误分类体系

为什么需要错误分类？

不是所有错误都应该被同样对待。有些是临时的（529），有些是配置问题（401），有些是用户错误（上下文超限）。正确分类是正确响应的前提。

```

classifyAPIError() 分类层次
├─ "aborted"          → 请求被用户中止
├─ "api_timeout"      → 连接超时
├─ "repeated_529"     → 连续 529 过载
├─ "capacity_off_switch" → Opus 紧急容量开关
├─ "rate_limit"       → 429 速率限制
├─ "server_overload"  → 529 服务器过载
├─ "prompt_too_long"  → 上下文超限
├─ "pdf_too_large"    → PDF 页面超限
├─ "pdf_password_protected" → PDF 加密
├─ "image_too_large"  → 图片尺寸超限
├─ "tool_use_mismatch" → tool_use/tool_result 不匹配
├─ "unexpected_tool_result" → 意外的 tool_result
├─ "duplicate_tool_use_id" → 重复的 tool_use ID
├─ "invalid_model"    → 无效模型名
├─ "credit_balance_low" → 信用额度不足
├─ "invalid_api_key"  → API 密钥无效
├─ "token_revoked"    → OAuth 令牌被撤销
├─ "auth_error"       → 401/403 认证错误
├─ "bedrock_model_access" → Bedrock 模型访问错误
├─ "server_error"     → 5xx 服务器错误
├─ "client_error"     → 4xx 客户端错误
├─ "ssl_cert_error"   → SSL 证书错误
├─ "connection_error" → 连接错误
└─ "unknown"         → 未分类

```

2.5 错误消息格式化

`getAssistantMessageFromError()` 将技术错误转换为用户可读的消息：

错误类型	用户消息	可操作建议
超时	"Request timed out"	检查网络连接和代理设置
529 重复	"Repeated 529 Overloaded errors"	等待后重试或切换模型
Opus 过载	"Opus is experiencing high load"	使用 <code>/model</code> 切换到 Sonnet
速率限制	显示配额和重置时间	等待或升级套餐
上下文超限	"Prompt is too long"	压缩对话或使用 <code>/clear</code>
API 密钥无效	"Not logged in · Please run <code>/login</code> "	重新登录
OAuth 令牌撤销	"OAuth token revoked · Please r..."	重新登录
SSL 错误	具体 SSL 错误描述	设置 <code>NODE_EXTRA_CA_CERTS</code>

2.6 关键文件索引

文件	核心职责
`src/services/api/withRetry.ts`	重试引擎、指数退避、模型回退、快速模式冷却
`src/services/api/errors.ts`	错误分类（25+ 类型）、用户消息生成
`src/services/api/errorUtils.ts`	连接错误提取、SSL 检测、HTML 清理
`src/services/api/logging.ts`	API 日志、分析数据上报
`src/utls/fastMode.ts`	快速模式冷却管理、额度超限处理
`src/services/claudeAiLimits.ts`	速率限制头解析、配额预警
`src/services/rateLimitMessages.ts`	速率限制用户消息生成

3. 上下文窗口管理

3.1 为什么需要上下文压缩？

大模型有固定的上下文窗口（如 200K tokens）。随着对话进行，消息越来越多，最终会触及上限。如果什么都不做，用户会收到"Prompt is too long"错误，会话被迫中断。自动压缩系统在达到上限前主动压缩对话，保持会话连续性。

3.2 自动压缩触发器

```
shouldAutoCompact() 检查
|
|— 来源检查: 如果是 compact 或 session_memory 自身 → 跳过 (防递归)
|— 功能开关: CONTEXT_COLLAPSE / REACTIVE_COMPACT 特性标记
|— 配置检查: DISABLE_COMPACT / DISABLE_AUTO_COMPACT
|
|— Token 计数检查:
|   |— 有效窗口大小 = min(模型上下文窗口 - max_output, CLAUDE_CODE_AUTO_COMPACT_WINDOW)
|   |— 压缩阈值 = 有效窗口大小 - 13,000 (AUTOCOMPACT_BUFFER_TOKENS)
|   |— 当前 token > 阈值 → 触发压缩
```

3.3 电路断路器 (Auto-Compact)



为什么压缩需要熔断器？ 压缩本身需要调用 API。如果压缩调用连续失败，每次对话都尝试压缩只会浪费 API 调用和等待时间。2026 年 3 月的数据显示：1,279 个会话出现了 50+ 次连续压缩失败（最多 3,272 次），每天浪费约 25 万次 API 调用。熔断器就是为此而生。


```

autoCompactIfNeeded()
├── 入口检查: consecutiveFailures >= 3 (MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES)
│   └── 是 → 直接返回 { wasCompacted: false }, 跳过压缩
├── 尝试压缩:
│   ├── 1. trySessionMemoryCompaction() → 基于会话内存的压缩
│   │   ├── 成功 → 重置 consecutiveFailures = 0
│   │   └── 失败 → 回退到传统紧凑压缩
│   └── 2. compactConversation() → 传统 API 压缩
│       ├── 成功 → 重置 consecutiveFailures = 0
│       └── 失败 (catch) → consecutiveFailures = prevFailures + 1
└── 状态传播:
    ├── 压缩结果 → query.ts 的 tracking 状态
    └── tracking → 下一次 autoCompactIfNeeded 调用的输入

```

熔断器状态的三个接触点：

接触点	操作	位置
入口	检查 `consecutiveFailures >= 3`, ...	`autoCompact.ts:257-265`
成功	重置 `consecutiveFailures = 0`	`autoCompact.ts:328-333`
失败	递增 `consecutiveFailures = prev ...`	`autoCompact.ts:334-351`

3.4 压缩重试机制

Prompt Too Long (PTL) 重试

为什么需要 PTL 重试？压缩的目的是减少 token 数量，但压缩请求本身可能因为对话太长而触发"prompt too long"错误。这就像用吸尘器清理被堵住的吸尘器——需要先手动清理一部分。

```

compactConversation() 的 PTL 重试
├── 调用 streamCompactSummary()
├── 收到 PROMPT_TOO_LONG 错误
│   ├── ptlAttempts <= 3 (MAX_PTL_RETRIES)
│   │   ├── truncateHeadForPTLRetry() → 丢弃最旧的 API 轮次
│   │   ├── 更新 forkContextMessages
│   │   └── 重试
│   └── ptlAttempts > 3 或无法丢弃
│       └── 抛出 ERROR_MESSAGE_PROMPT_TOO_LONG
└── 成功 → 返回压缩结果

```

流式重试

```
streamCompactSummary() 的流式重试
```

- | 尝试 forked-agent 路径（共享提示缓存）
 - | 成功 → 返回结果
 - | 失败 → 回退到直接流式
- | 直接流式路径（最多 MAX_COMPACT_STREAMING_RETRIES = 2 次）
 - | 受 GrowthBook 开关 'tengu_compact_streaming_retry' 控制
 - | 全部失败 → 抛出 ERROR_MESSAGE_INCOMPLETE_RESPONSE

3.5 会话内存压缩 (Session Memory Compact)

为什么需要会话内存压缩？ 传统压缩是"用 API 总结对话"——每次压缩都要调用 API，成本高且可能失败。会话内存是后台持续提取的知识库，压缩时直接使用已有的知识，无需额外 API 调用——零成本压缩。

```
trySessionMemoryCompaction()
```

- | 检查: shouldUseSessionMemoryCompaction() → 功能开关 + 环境变量
- | 等待: 正在进行的会话内存提取完成 (15 秒超时, 1 秒轮询)
- | 获取: lastSummarizedMessageId → 上次提取的位置
- | 读取: 会话内存文件 → 检查是否存在且非空
- | 计算保留消息:
 - | 从 lastSummarizedMessageId 开始
 - | 向后扩展到满足 minTokens (10K) 和 minTextBlockMessages (5 条)
 - | 上限 maxTokens (40K)
 - | 调整 API 不变量 (不拆分 tool_use/tool_result 对)
 - | 过滤掉旧的紧凑边界标记
- | 构建压缩结果:
 - | 截断过大的会话内存段
 - | 从截断的内存构建紧凑的摘要消息
 - | 创建边界标记
- | 验证: 压缩后 token 数是否超过 autoCompactThreshold
 - | 超过 → 返回 null (回退到传统压缩)
 - | 未超过 → 返回压缩结果

3.6 关键文件索引

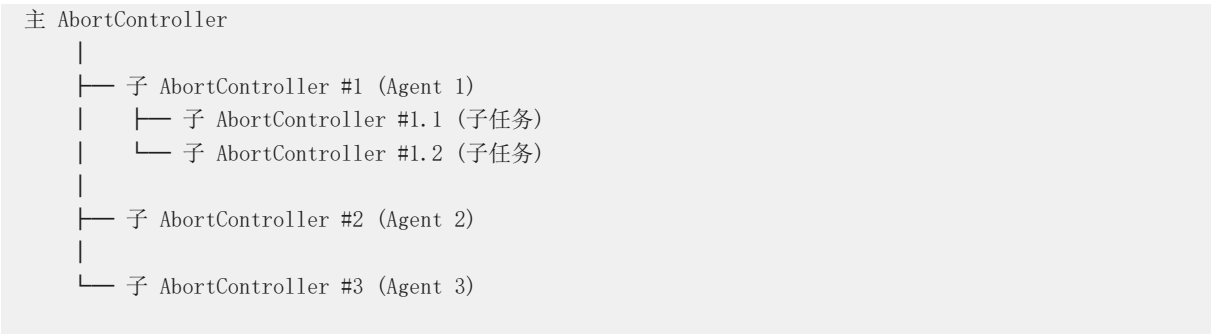
文件	核心职责
`src/services/compact/autoCompact.ts`	自动压缩触发、电路断路器 (3 次失败)
`src/services/compact/compact.ts`	传统压缩引擎、PTL 重试 (3 次)
`src/services/compact/sessionMemoryCompact.ts`	基于会话内存的零成本压缩
`src/services/compact/postCompactCleanup.ts`	压缩后状态清理
`src/services/SessionMemory/sessionMemory.ts`	后台非阻塞知识提取

4. Agent 编排与隔离

4.1 为什么需要 Agent 隔离?

当 Claude Code 启动子 Agent 执行任务时，子 Agent 的失败不应该影响主任务。例如，一个 Agent 在写代码时触发另一个 Agent 做代码审查，审查 Agent 的超时不应该打断写代码的进程。

4.2 子 AbortController 隔离



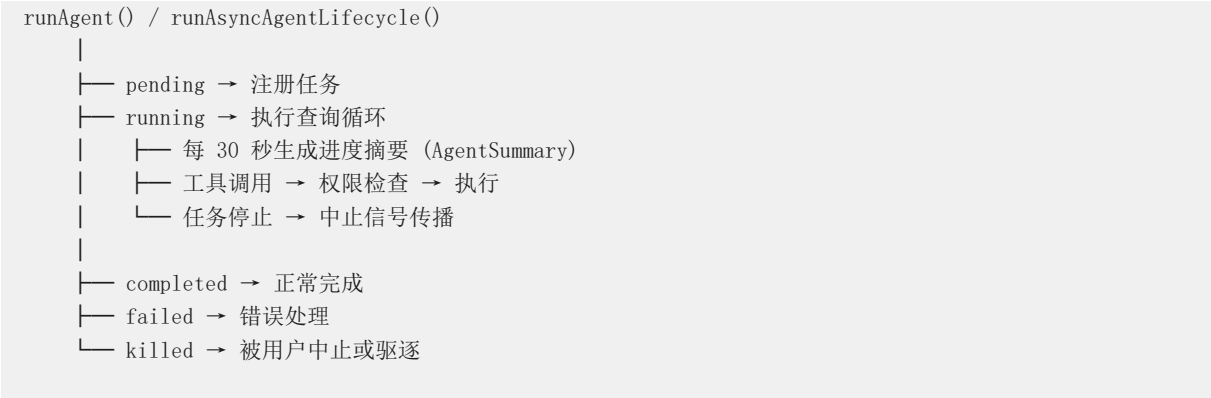
设计特点:

- 父控制器中止时，所有子控制器自动中止（级联中止）
- 子控制器中止不会传播到父控制器（单向隔离）
- 使用 WeakRef 持有父子关系：如果子控制器被丢弃，父控制器不会持有死引用
- 子控制器中止时自动清理事件监听器，防止内存泄漏

4.3 Forked Agent 上下文隔离



4.4 Agent 任务生命周期



4.5 任务驱逐机制

为什么需要任务驱逐？ 如果 Agent 执行时间过长、占用资源过多，系统需要有能力回收资源。

```
evictTerminalTask() 的驱逐流程
├─ 标记任务为已完成/失败/已中止
├─ 发送中止信号
├─ 清理注册的任务状态
└─ 通知用户任务已被驱逐
```

4.6 协调器模式

```
coordinatorMode.ts 的五阶段方法
├─ 阶段 1: 规划 → 分析任务，制定计划
├─ 阶段 2: 分配 → 将子任务分配给 Worker Agent
├─ 阶段 3: 执行 → Worker 并行执行
│   └─ 每个 Worker 独立中止控制器
│       └─ 单个 Worker 失败不影响其他 Worker
├─ 阶段 4: 汇总 → 收集结果
└─ 阶段 5: 输出 → 输出最终结果
```

4.7 关键文件索引

文件	核心职责
`src/tools/AgentTool/AgentTool.tsx`	Agent 核心编排 (~1900 行)
`src/tools/AgentTool/runAgent.ts`	Agent 执行循环、30 秒进度摘要
`src/tools/AgentTool/resumeAgent.ts`	已停止 Agent 恢复
`src/utils/forkedAgent.ts`	上下文隔离、子 AbortController
`src/utils/abortController.ts`	子 AbortController 工厂、WeakRef 管理
`src/coordinator/coordinatorMode.ts`	五阶段协调器模式
`src/utils/sequential.ts`	防竞态队列序列化

5. 错误处理与恢复

5.1 三级错误处理体系

三级错误处理体系		
第一级：API 错误 (withRetry.ts + errors.ts)		
重试 → 回退 → 分类 → 格式化 → 用户消息		
第二级：工具执行错误 (toolExecution.ts)		
AbortError	→ 静默忽略	
ShellError	→ 格式化输出，不记录错误日志	
其他错误	→ 记录错误日志 + 分析数据	
第三级：进程级错误 (gracefulShutdown.ts)		
uncaughtException	→ 记录日志，不退出进程	
unhandledRejection	→ 记录日志，不退出进程	
SIGINT/SIGTERM/SIGHUP	→ 优雅关闭	

5.2 工具执行错误分类

executeToolUse() 的 catch 块



5.3 对话恢复机制

为什么需要对话恢复？

会话可能因为网络断开、进程崩溃、用户关闭终端等意外情况中断。恢复机制让用户可以从中断处继续，而不是重新开始。

中断检测

```

deserializeMessagesWithInterruptDetection()
├── 1. 遗留类型迁移 → 转换旧的 attachment 类型
├── 2. 权限模式验证 → 剥离无效的 permissionMode
├── 3. 过滤管道:
│   ├── filterUnresolvedToolUses()
│   │   ├── → 移除 tool_use 没有对应 tool_result 的 assistant 消息
│   │   └── → 处理场景: 会话在工具执行中中断
│   ├── filterOrphanedThinkingOnlyMessages()
│   │   ├── → 移除没有对应非 thinking 内容的纯 thinking 消息
│   │   └── → 处理场景: 流式 thinking 作为独立消息到达
│   └── filterWhitespaceOnlyAssistantMessages()
│       ├── → 移除仅含空白的 assistant 消息
│       └── → 合并相邻的 user 消息保持交替角色
├── 4. 中断检测 (detectTurnInterruption):
│   ├── 最后消息是 assistant → 正常完成
│   ├── 最后消息是 user + 元/摘要 → 正常
│   ├── 最后消息是 user + tool_result
│   │   ├── 是终端工具 → 正常
│   │   └── 否 → interrupted_turn
│   ├── 最后消息是 user + 文本 → interrupted_prompt
│   └── 最后消息是 attachment → interrupted_turn
└── 5. 合成续接:
    ├── interrupted_turn → 注入 "Continue from where you left off."
    └── 注入 assistant sentinel → 确保 API 有效

```

会话恢复流程

```

loadConversationForResume()
├── 1. 确定来源:
│   ├── 无指定 → 加载最近的日志 (跳过活跃的 --bg/daemon 会话)
│   ├── --resume <id> → 加载指定会话
│   └── --continue → 从文件加载
├── 2. 恢复状态:
│   ├── 文件历史记录
│   ├── 已调用的技能
│   ├── 会话计划
│   ├── 工作目录 (worktree)
│   └── 上下文折叠状态
├── 3. 恢复配置:
│   ├── Agent 类型和模型覆盖
│   ├── 协调器模式匹配
│   └── 权限模式
└── 4. 注入会话启动钩子 → 恢复 CLAUDE.md 等上下文

```

5.4 错误日志系统

```
errorLogSink.ts
|
├─ 初始化: attachErrorLogSink() → 惰性注册
├─ 写入路径: ~/.claude/errors/<date>.jsonl
├─ 缓冲写入: 1 秒刷新间隔, 50 条最大缓冲
├─ 清理注册: registerCleanup() → 关闭时刷新
└─ 仅 Ant 员工: 包含完整错误详情 (cwd, userId, sessionId, version)
```

5.5 关键文件索引

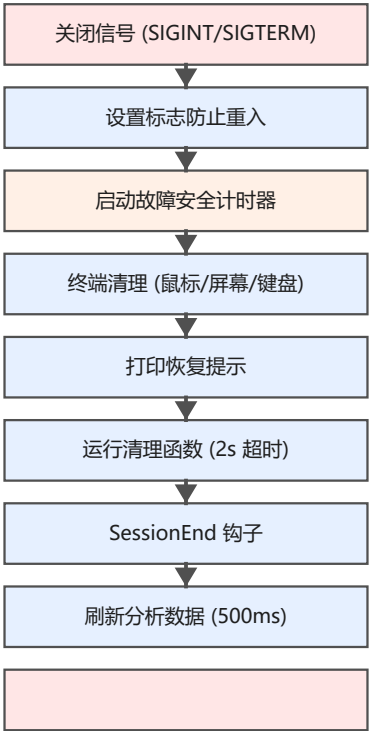
文件	核心职责
`src/utils/conversationRecovery.ts`	中断检测、消息过滤、合成续接
`src/utils/sessionRestore.ts`	完整状态恢复引擎
`src/utils/sessionState.ts`	idle/running/requires_action 状态机
`src/utils/sessionStorage.ts`	JSONL 会话持久化
`src/utils/errorLogSink.ts`	缓冲 JSONL 错误日志
`src/utils/errors.ts`	核心错误类 (AbortError, ShellError 等)
`src/utils/toolErrors.ts`	工具错误格式化与截断
`src/services/tools/toolExecution.ts`	工具执行错误分类与处理
`src/utils/debug.ts`	调试日志系统

6. 进程级保障

6.1 为什么需要进程级保障？

API 重试和错误处理只能应对"预期"的异常。但进程可能被用户意外关闭（SIGINT）、系统杀死（SIGTERM）、终端崩溃（orphan 检测），这些情况需要更底层的保护机制。

6.2 优雅关闭流程



forceExit() → SIGKILL 回退

max(5s, 钩子超时 + 3.5s)



```

gracefulShutdown(exitCode, reason)
|
| ── 守卫: shutdownInProgress === true → 立即返回 (防重入)
|
| ── 设置 shutdownInProgress = true
|
| ── 武装故障安全计时器:
|   ── 预算 = max(5s, sessionEndTimeoutMs + 3.5s)
|   ── 到期 → cleanupTerminalModes() → printResumeHint() → forceExit(code)
|
| ── 阶段 1: 终端清理 (同步)
|   ── 禁用鼠标追踪
|   ── 退出 alt screen
|   ── 消耗 stdin 残留事件
|   ── 禁用键盘扩展协议
|   ── 显示光标
|   ── 清除终端标题/进度条
|
| ── 阶段 2: 打印恢复提示
|   ── "claude --resume <sessionId>"
|
| ── 阶段 3: 清理函数 (2 秒超时)
|   ── runCleanupFunctions() → 50+ 模块并行清理
|
| ── 阶段 4: 会话结束钩子
|   ── executeSessionEndHooks(reason) → 超时控制
|
| ── 阶段 5: 分析数据上报 (500ms 超时)
|   ── Promise.all([shutdownPEventLogging, shutdownDatadog]) vs 500ms sleep
|
| ── 阶段 6: 强制退出
|   ── process.exit(exitCode)
|   ── EIO 错误 → SIGKILL 回退
|   ── Error('unreachable') → 永不到达

```

6.3 故障安全计时器

为什么需要故障安全计时器？如果某个清理函数挂起（比如 MCP 服务器无响应），进程可能永远无法退出。故障安全计时器就像一个“最后期限”——到了时间不管清理是否完成，强制退出。

```

// 故障安全计时器逻辑
const failsafeTimer = setTimeout(() => {
  cleanupTerminalModes() // 确保终端可用
  printResumeHint()      // 打印恢复提示
  forceExit(exitCode)     // 强制退出
}, Math.max(5000, sessionEndTimeoutMs + 3500))

// 正常关闭时清除计时器
if (failsafeTimer) clearTimeout(failsafeTimer)

```

计时器被 `.unref()` 化，不会阻止进程自然退出。

6.4 孤儿进程检测 (macOS)

为什么需要孤儿检测？macOS 在关闭终端窗口时可能不发送 `SIGHUP` 信号，导致进程变成孤儿但继续运行。

```
// 每 30 秒检查 TTY 状态
const orphanCheckInterval = setInterval(() => {
  if (!process.stdout.writable || !process.stdin.readable) {
    // TTY 已被撤销 → 触发优雅关闭
    gracefulShutdown(129)
  }
}, 30000) // 30 秒间隔
```

6.5 超时管理

组件	超时时间	用途
Bash 工具默认	2 分钟	`BASH_DEFAULT_TIMEOUT_MS`
Bash 工具最大	10 分钟	`BASH_MAX_TIMEOUT_MS`
清理函数	2 秒	`runCleanupFunctions()`
分析上报	500ms	`shutdownPEventLogging + shut...
会话结束钩子	可配置	`sessionEndTimeoutMs`
故障安全	max(5s, 钩子 + 3.5s)	强制退出截止
空闲超时	可配置	`CLAUDE_CODE_EXIT_AFTER_ST...

6.6 防止睡眠 (macOS)

为什么需要防止睡眠?

长时间运行的任务（如代码库分析）可能耗时数十分钟。如果电脑在这期间进入睡眠，任务会中断。

```
startPreventSleep()
├─ 引用计数 +1
├─ 首次调用 → 启动 caffeinate -i -t 300
│   └─ -i: 防止空闲睡眠
│   └─ -t 300: 5 分钟后自动退出（自愈机制）
│       └─ 每 4 分钟重启一次，保持连续覆盖
└─ caffeinateProcess.unref() → 不阻止 Node 退出

stopPreventSleep()
├─ 引用计数 -1
└─ 计数归零 → 杀死 caffeinate 进程
```

6.7 关键文件索引

文件	核心职责
`src/utils/gracefulShutdown.ts`	信号处理、分阶段关闭、故障安全计时器
`src/utils/cleanupRegistry.ts`	全局清理注册表 (50+ 模块)
`src/utils/abortController.ts`	子 AbortController 工厂、WeakRef
`src/utils/combinedAbortSignal.ts`	自定义超时信号（避免 Bun 内存泄漏）
`src/utils/timeouts.ts`	超时配置（默认 2min，最大 10min）
`src/utils/idleTimeout.ts`	空闲超时管理
`src/services/preventSleep.ts`	caffeinate 进程管理、引用计数

7. 熔断与降级模式全景

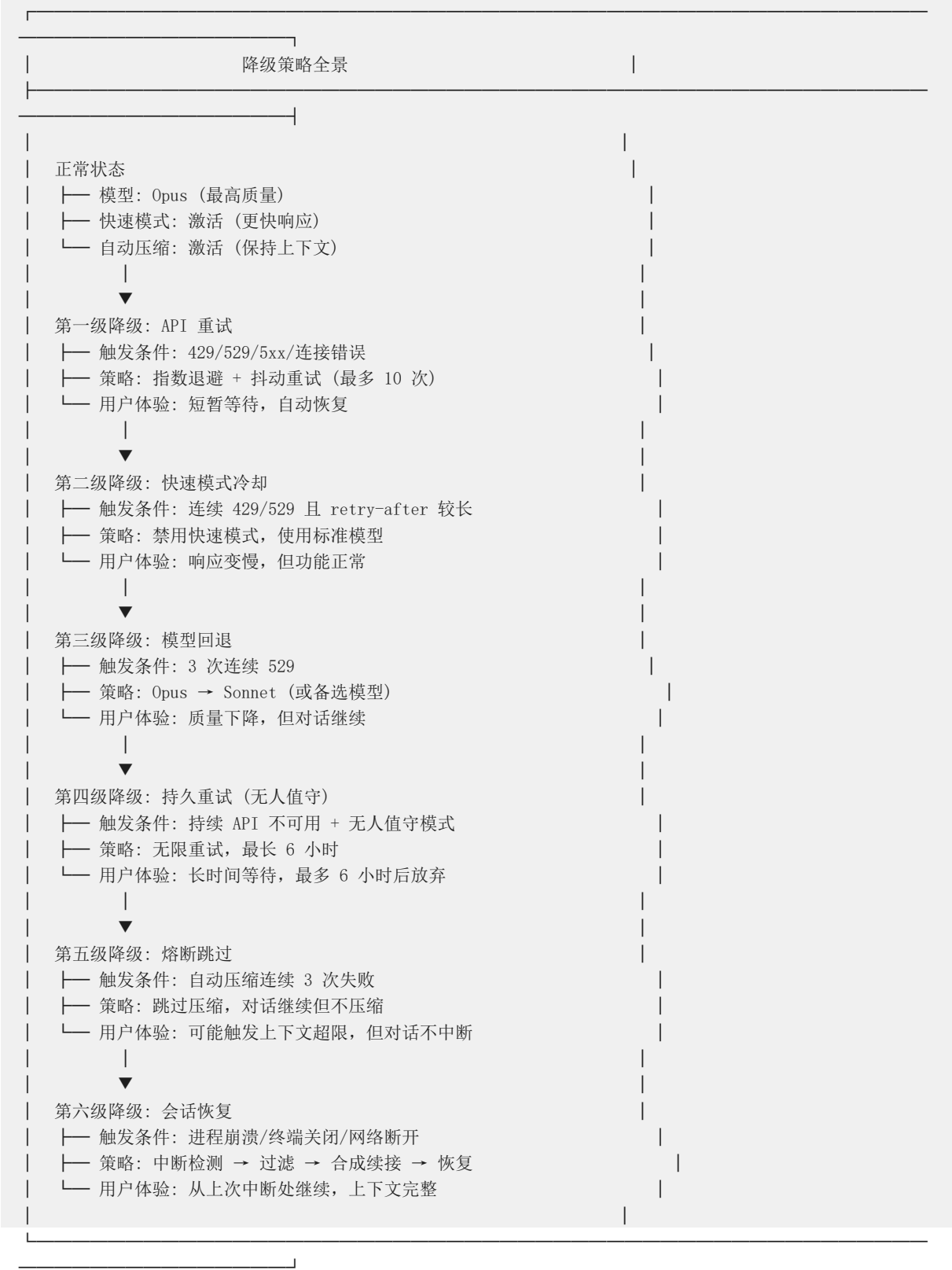
7.1 熔断器一览



Claude Code 中实现了多个熔断器，分别保护不同资源：

熔断器	位置	阈值	触发动作	恢复方式
自动压缩熔断	<code>`autoCompact.ts`</code>	3 次连续失败	跳过所有后续自动压缩	需要新会话
529 模型回退	<code>`withRetry.ts`</code>	3 次连续 529	切换到备选模型	成功后自动恢复
快速模式冷却	<code>`fastMode.ts`</code>	收到 429/529	禁用快速模式（默...	冷却到期自动恢复
快速模式禁用	<code>`fastMode.ts`</code>	收到 "not enabled"	永久禁用快速模式	不可恢复
快速模式额度超限	<code>`fastMode.ts`</code>	收到 overage 拒绝	永久禁用快速模式	不可恢复
CCR 认证熔断	<code>`ccrClient.ts`</code>	10 次连续认证失败	停止认证重试	需要新会话
Bridge 初始化熔断	<code>`useReplBridge.tsx`</code>	3 次连续初始化失败	停止桥接重试	需要新会话
CCR 会话熔断	<code>`ccrSession.ts`</code>	5 次连续失败	停止会话重试	需要新会话
快速模式自动熔断	<code>`permissionSetup.ts`</code>	GrowthBook 开关...	禁用自动模式	需要远程配置变更

7.2 降级策略全景



7.3 错误恢复的时间线

时间线

- |
- |— 0ms: API 请求开始
- |
- |— ~500ms: 请求超时或收到错误响应
- |
- | |— 529 错误:
 - | | |— 第 1 次: 等待 ~500ms + 抖动 → 重试
 - | | |— 第 2 次: 等待 ~1s + 抖动 → 重试
 - | | |— 第 3 次: 等待 ~2s + 抖动 → 重试
 - | | |— 第 3+ 次: 触发模型回退或放弃
- |
- | |— 429 快速模式错误:
 - | | |— retry-after < 20s: 等待后保留快速模式重试
 - | | |— retry-after >= 20s: 触发 30min 冷却
- |
- | |— 连接超时:
 - | | |— 第 1 次重试: ~500ms
 - | | |— 第 2 次重试: ~1s
 - | | |— ...
- |
- | |— 上下文超限 (400):
 - | | |— 调整 max_tokens 后重试
 - | | |— 失败后触发自动压缩 (如果尚未触发)
- |
- |— ~30s: 持久重试模式的心跳输出
- |
- |— ~30min: 快速模式冷却到期, 自动恢复
- |
- |— ~6h: 持久重试模式绝对上限 (CLAUDE_CODE_UNATTENDED_RETRY)

7.4 设计模式总结

模式	用途	实现方式
电路断路器	防止连续失败放大问题	计数器 + 阈值检查 + 跳过
指数退避	避免重试风暴	<code>BASE_DELAY_MS * 2^(attempt-1)</code>
抖动	分散重试时间	<code>random(0, 0.25 * BASE_DELAY_...</code>
回退链	逐步降级服务质量	快速模式 → 标准模型 → 备选模型
心跳	保持长等待连接活跃	30 秒间隔输出系统消息
故障安全	防止进程挂起	武装超时计时器, 到期强制退出
引用计数	共享资源管理	<code>start/stopPreventSleep</code> 的引用计数
WeakRef	防止内存泄漏	子 AbortController 的 WeakRef 持有
哨兵值	熔断器状态传播	<code>tracking.consecutiveFailures</code> 的...
隔离舱壁	限制故障影响范围	子 AbortController 的单向传播

7.5 关键文件索引

文件	核心职责
`src/services/api/withRetry.ts`	重试引擎、指数退避、模型回退、快速模式冷却
`src/services/api/errors.ts`	错误分类（25+ 类型）、用户消息生成
`src/services/api/errorUtils.ts`	连接错误提取、SSL 检测
`src/services/compact/autoCompact.ts`	自动压缩熔断器
`src/utils/gracefulShutdown.ts`	优雅关闭、故障安全计时器、孤儿检测
`src/utils/abortController.ts`	子 AbortController 工厂、WeakRef
`src/utils/conversationRecovery.ts`	中断检测、消息过滤、会话恢复
`src/utils/fastMode.ts`	快速模式冷却管理
`src/utils/sequential.ts`	防竞态队列序列化
`src/utils/cleanupRegistry.ts`	全局清理注册表
`src/utils/combinedAbortSignal.ts`	自定义超时信号
`src/services/preventSleep.ts`	防止系统睡眠
`src/services/tools/toolExecution.ts`	工具执行错误分类
`src/utils/toolErrors.ts`	工具错误格式化

附录 A：参考数据

关键配置参数总表

参数	文件	默认值	可覆盖
`DEFAULT_MAX_RETRIES`	`withRetry.ts`	10	`CLAUDE_CODE_MAX_R...
`MAX_529_RETRIES`	`withRetry.ts`	3	否
`BASE_DELAY_MS`	`withRetry.ts`	500ms	否
`PERSISTENT_MAX_BAC...	`withRetry.ts`	300,000ms (5min)	否
`PERSISTENT_RESET_CA...	`withRetry.ts`	21,600,000ms (6hr)	否
`HEARTBEAT_INTERVAL...	`withRetry.ts`	30,000ms (30s)	否
`MIN_COOLDOWN_MS`	`withRetry.ts`	600,000ms (10min)	否
`MAX_CONSECUTIVE_A...	`autoCompact.ts`	3	否
`AUTOCOMPACT_BUFF...	`autoCompact.ts`	13,000	`CLAUDE_AUTOCOMPA...
`MAX_PTL_RETRIES`	`compact.ts`	3	否
`MAX_COMPACT_STRE...	`compact.ts`	2	否
`DEFAULT_TIMEOUT_MS`	`timeouts.ts`	120,000ms (2min)	`BASH_DEFAULT_TIME...
`MAX_TIMEOUT_MS`	`timeouts.ts`	600,000ms (10min)	`BASH_MAX_TIMEOUT_...
`MAX_CONSECUTIVE_A...	`ccrClient.ts`	10	否
`MAX_CONSECUTIVE_I...	`useReplBridge.tsx`	3	否
`MAX_CONSECUTIVE_F...	`ccrSession.ts`	5	否

编译标记控制的特性

特性标记	默认值	影响
`UNATTENDED_RETRY`	编译时	启用持久重试模式
`PROACTIVE`	编译时	启用主动功能
`KAIROS`	编译时	启用 KAIROS 特定功能
`CONTEXT_COLLAPSE`	编译时	启用上下文折叠

GrowthBook 远程配置开关

开关	影响
`tengu_session_memory`	会话内存提取
`tengu_sm_compact`	会话内存压缩
`tengu_compact_streaming_retry`	压缩流式重试
`tengu_compact_cache_prefix`	压缩提示缓存共享
`tengu_sm_compact_config`	会话内存压缩配置 (minTokens/maxTokens/minText...
`tengu_auto_mode_config`	自动模式配置 (含熔断开关)

本报告由 codebase-deep-analysis skill 自动生成，基于 Claude Code 源码快照的静态分析。